

Firestore Auth Backend Guide

A reusable AI implementation manual for Flutter projects

Backend only: Firestore Auth, domain/data layers, core helpers, validation, remembered email, email verification, password reset, Google sign-in, deep-link return, and hosted reset page guidance.

Generated from docs/guides/firestore_auth_backend_ai_guide.md

Firestore Auth Backend Guide for Flutter AI Builders

Version: 1.0 Last verified: 2026-05-10 Scope: Flutter backend/auth module only Excludes: presentation screens, routing UI, Riverpod/GetX/BLoC/state management, widget code, tests

How To Use This Guide

Give this document to an AI when you want it to implement a clean Firestore Authentication backend in a new Flutter project.

The AI must:

- Build only the authentication backend, domain, data, and core helper files.
- Avoid adding screens, widgets, routes, controllers, providers, Riverpod, GetX, BLoC, or UI state.
- Use Firestore Auth for email/password, Google sign-in, email verification, reset password, sign-out, auth stream, profile mapping, and secure error handling.
- Keep Firestore SDK code inside the data layer.
- Keep business contracts in the domain layer.
- Keep validation, result, network, storage, and exception helpers in core/shared folders.
- Use manual construction or a simple factory. Do not use injectable or a dependency-injection generator.

Official references used:

- Firestore Flutter setup: <https://firebase.google.com/docs/flutter/setup>
- Firestore password auth for Flutter: <https://firebase.google.com/docs/auth/flutter/password-auth>
- Firestore federated/social sign-in for Flutter: <https://firebase.google.com/docs/auth/flutter/federated-auth>
- Firestore manage users for Flutter: <https://firebase.google.com/docs/auth/flutter/manage-users>

Target Feature Set

Implement these backend capabilities:

1. Initialize Firestore from `firebase_options.dart`.
2. Sign up with email, password, and display name.
3. Send verification email after sign-up.
4. Block email/password login if the email is not verified.
5. Resend verification email.
6. Reload current user and check whether email became verified.
7. Sign in with email and password.
8. Sign in with Google.
9. Send password reset email.

10. Sign out from Firebase and Google.
11. Listen to auth state changes.
12. Map Firebase users to a pure domain `UserEntity`.
13. Save only remembered email locally when the user chooses "remember me".
14. Validate email, password, name, and optional phone input with reusable helpers.
15. Map Firebase errors into stable custom exceptions and failures.
16. Check network connectivity before remote auth calls.
17. Provide optional Firebase action-code settings for email verification and password reset links.
18. Support the same production flow as the Teka Luxe project:

sign-up -> send Firebase verification email -> pending verification -> verified success -> login/home.

19. Support forgot password:

app sends Firebase reset email -> user opens custom web reset page -> page updates Firebase password -> page redirects back to the app.

20. Register the app deep link `teka-luxe://auth-callback` or the equivalent app scheme for the new project.

Full Auth Flow Like This Project

This is the exact backend flow the AI should recreate in another project.

```
APP START
+-- Firebase.initializeApp(...)
+-- authStateChanges()
    |-- null -> unauthenticated flow
+-- user exists
    |-- email/password user and emailVerified=false -> verification pending
+-- Google user or verified email user -> authenticated flow

SIGN UP
+-- validate name/email/password/phone if used
+-- createUserWithEmailAndPassword
+-- updateDisplayName
    +-- sendEmailVerification(actionCodeSettings optional)
    +-- return UserEntity with emailVerified=false

VERIFY EMAIL
+-- user opens Firebase email link
    |-- default Firebase handler, or
+-- custom action page applies Firebase action code
+-- app reloads current user
    |-- verified=false -> stay pending
+-- verified=true -> show success then allow authenticated flow

LOGIN
+-- validate email/password
+-- signInWithEmailAndPassword
+-- reload user
    |-- emailVerified=false -> signOut and return emailNotVerified failure
+-- emailVerified=true -> return UserEntity

GOOGLE LOGIN
+-- GoogleSignIn -> Firebase credential -> signInWithCredential -> return UserEntity

FORGOT PASSWORD
+-- validate email
+-- sendPasswordResetEmail(actionCodeSettings optional)
+-- Firebase email link opens custom reset web page
+-- confirmPasswordReset(auth, oobCode, newPassword)
    +-- show success timer
    +-- redirect to app deep link

REMEMBER ME
+-- if checked save normalized email only
+-- if unchecked remove remembered email
```

Step-By-Step Build Order For The AI

Follow this order. Do not jump into UI or state-management work.

Step 1 - Add Dependencies

```
flutter pub add firebase_core firebase_auth google_sign_in connectivity_plus
shared_preferences
dart pub global activate flutterfire_cli
flutterfire configure
```

Select Android and iOS when prompted. Add web only if the project needs a hosted Firebase action page inside Flutter web. For the static reset page approach, a plain HTML page is enough.

Step 2 - Configure Firebase Console

Enable:

- Authentication > Sign-in method > Email/Password.
- Authentication > Sign-in method > Google.
- Authentication > Templates > Email address verification.
- Authentication > Templates > Password reset.

For Google sign-in:

- Android: add SHA-1 and SHA-256 fingerprints.
- iOS: confirm the reversed client ID is present in the generated `GoogleService-Info.plist` if the selected package requires it.

Step 3 - Add Firebase Initialization

Initialize Firebase before composing the backend.

```
await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);
```

Step 4 - Add Core Helpers

Create:

- `core/utils/result.dart`
- `core/errors/auth_exception.dart`
- `core/errors/auth_failure.dart`
- `core/utils/auth_validators.dart`
- `core/network/network_info.dart`
- `core/storage/remembered_email_storage.dart`

- `core/utils/auth_action_code_settings.dart`
- `core/utils/auth_cooldown.dart`

Step 5 - Add Domain Layer

Create:

- `UserEntity`
- `IAuthRepository`
- one use case per auth action

The domain layer must not import `FirebaseAuth`, except when passing optional `ActionCodeSettings` is intentionally accepted by the project. If strict clean architecture is required, wrap `ActionCodeSettings` in a project-defined settings object.

Step 6 - Add Data Layer

Create:

- `UserModel`
- `AuthRemoteDataSource`
- `FirebaseAuthRemoteDataSource`
- `FirebaseAuthRepository`

Only this layer should call Firebase SDK methods.

Step 7 - Add Backend Factory

Create a manual factory that wires `FirebaseAuth`, `Connectivity`, `SharedPreferences`, repository, and use cases. Do not use injectable.

Step 8 - Add Native Deep Link Return

If password reset or email verification should return to the app, register a custom URL scheme.

Replace `teka-luxe` with the new app scheme.

Android file: `android/app/src/main/AndroidManifest.xml`

```
<intent-filter>
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.DEFAULT"/>
  <category android:name="android.intent.category.BROWSABLE"/>
  <data
    android:scheme="teka-luxe"
    android:host="auth-callback"/>
</intent-filter>
```

This intent filter belongs inside the main `<activity>`.

iOS file: `ios/Runner/Info.plist`

```
<key>CFBundleURLTypes</key>
<array>
  <dict>
    <key>CFBundleURLName</key>
    <string>teka-luxe.auth-callback</string>
    <key>CFBundleURLSchemes</key>
    <array>
      <string>teka-luxe</string>
    </array>
  </dict>
</array>
```

Step 9 - Add Firebase Email Templates

Firebase templates are configured manually in Firebase Console. They are not pushed by Flutter code.

Use `%LINK%` for Firebase's generated action link and `%EMAIL%` for the user's email.

Verification template:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta content="width=device-width, initial-scale=1.0" name="viewport" />
    <title>Verify your email</title>
  </head>
  <body style="margin:0;padding:0;background:#fafbff;color:#080a12;font-family:Arial,Helvetica,sans-serif;">
    <table role="presentation" width="100%" cellpadding="0" cellspacing="0" style="background:#fafbff;border-collapse:collapse;">
      <tr>
        <td align="center" style="padding:42px 18px;">
          <table role="presentation" width="560" cellpadding="0" cellspacing="0" style="width:560px;max-width:100%;border-collapse:collapse;">
            <tr>
              <td align="center" style="padding:0 0 22px;">
                <div style="color:#315cff;font-size:13px;font-weight:800;letter-spacing:.22em;text-transform:uppercase;">APP NAME</div>
              </td>
            </tr>
            <tr>
              <td style="background:#fff;border:1px solid #d7dee9;border-radius:34px;padding:44px 40px;box-shadow:0 18px 42px rgba(49,92,255,.08);text-align:center;">
                <h1 style="margin:0;color:#080a12;font-family:Georgia,'Times New Roman',serif;font-size:42px;line-height:46px;font-weight:800;">Verify Your Email</h1>
                <p style="margin:18px 0 0;color:#323746;font-size:16px;line-height:27px;">Confirm %EMAIL% to finish setting up your account.</p>
                <a href="%LINK%" style="display:inline-block;min-width:220px;margin-top:32px;background:#315cff;border-radius:20px;color:#fff;font-size:14px;font-weight:800;letter-spacing:.08em;line-height:56px;text-align:center;text-decoration:none;text-transform:uppercase;">Verify Email</a>
              </td>
            </tr>
            <tr>
              <td align="center" style="padding:22px 22px 0;">
                <p style="margin:0;color:#71798a;font-size:12px;line-height:19px;">If you did not create this account, you can ignore this email.</p>
              </td>
            </tr>
          </table>
        </td>
      </tr>
    </table>
  </body>
</html>
```

Password reset template:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
```

```

<meta content="width=device-width, initial-scale=1.0" name="viewport" />
<title>Reset your password</title>
</head>
<body style="margin:0;padding:0;background:#fafbff;color:#080a12;font-family:Arial,Helvetica,sans-serif;">
<table role="presentation" width="100%" cellpadding="0" cellspacing="0" style="background:#fafbff;border-collapse:collapse;">
<tr>
<td align="center" style="padding:42px 18px;">
<table role="presentation" width="560" cellpadding="0" cellspacing="0" style="width:560px;max-width:100%;border-collapse:collapse;">
<tr>
<td align="center" style="padding:0 0 22px;">
<div style="color:#315cff;font-size:13px;font-weight:800;letter-spacing:.22em;text-transform:uppercase;">APP NAME</div>
</td>
</tr>
</tr>
<tr>
<td style="background:#fff;border:1px solid #d7dee9;border-radius:34px;padding:44px 40px;box-shadow:0 18px 42px rgba(49,92,255,.08);text-align:center;">
<h1 style="margin:0;color:#080a12;font-family:Georgia,'Times New Roman',serif;font-size:42px;line-height:46px;font-weight:800;">Reset Your Password</h1>
<p style="margin:18px 0 0;color:#323746;font-size:16px;line-height:27px;">We received a password reset request for %EMAIL%. Tap the button below to choose a new password.</p>
<a href="%LINK%" style="display:inline-block;min-width:220px;margin-top:32px;background:#315cff;border-radius:20px;color:#fff;font-size:14px;font-weight:800;letter-spacing:.08em;line-height:56px;text-align:center;text-decoration:none;text-transform:uppercase;">Reset Password</a>
</td>
</tr>
<tr>
<td align="center" style="padding:22px 22px 0;">
<p style="margin:0;color:#71798a;font-size:12px;line-height:19px;">If you did not request a password reset, you can ignore this email.</p>
</td>
</tr>
</table>
</td>
</tr>
</table>
</body>
</html>

```

Step 10 - Add Custom Reset Password Web Page

If the project uses a hosted reset page like this project, create a static file such as:

netlify/reset-password/index.html

Configure Firebase Password Reset template action URL to the hosted page, for example:

https://your-reset-page.example.com/

The page must read Firebase action-code query parameters:

```

mode=resetPassword
oobCode=<firebase-action-code>
continueUrl=<optional-app-link>

```

Minimal working reset page logic:

```

<script type="module">
import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.10.0/firebase-app.js";
import {
confirmPasswordReset,
getAuth,
verifyPasswordResetCode,
} from "https://www.gstatic.com/firebasejs/11.10.0/firebase-auth.js";

const firebaseConfig = {
apiKey: "YOUR_API_KEY",
authDomain: "YOUR_PROJECT.firebaseio.com",
projectId: "YOUR_PROJECT",
storageBucket: "YOUR_PROJECT.firebaseio.com",

```

```

    messagingSenderId: "YOUR_SENDER_ID",
  };

  const app = initializeApp(firebaseConfig);
  const auth = getAuth(app);

  const url = new URL(window.location.href);
  const mode = url.searchParams.get("mode");
  const oobCode = url.searchParams.get("oobCode");
  const continueUrl =
    url.searchParams.get("continueUrl") || "teka-luxe://auth-callback";

  if (mode !== "resetPassword" || !oobCode) {
    throw new Error("This reset link is invalid or expired.");
  }

  await verifyPasswordResetCode(auth, oobCode);

  async function submitNewPassword(newPassword) {
    await confirmPasswordReset(auth, oobCode, newPassword);
    startAppRedirect(continueUrl);
  }

  function startAppRedirect(targetUrl) {
    let secondsLeft = 3;
    const countdown = document.querySelector("#redirect-countdown");
    const action = document.querySelector("#redirect-action");
    action.href = targetUrl;
    countdown.textContent = `Opening app in ${secondsLeft} seconds...`;

    const timer = window.setInterval(() => {
      secondsLeft -= 1;
      if (secondsLeft <= 0) {
        window.clearInterval(timer);
        window.location.href = targetUrl;
      } else {
        countdown.textContent = `Opening app in ${secondsLeft} seconds...`;
      }
    }, 1000);
  }
}
</script>

```

The visible page should include:

- New password field.
- Confirm password field.
- Professional eye toggle using `aria-pressed`.
- Save button with loading state.
- Success message.
- Redirect countdown.
- Manual "Open App" fallback link.

Step 11 - Add Resend Verification Timer

Use `AuthCoooldown` from this guide. The backend does not render a timer, but it provides the rule. The presentation layer can show the countdown.

Expected behavior:

- User taps resend.
- Backend calls `sendVerificationEmail`.
- `AuthCoooldown.markTriggered()` is called only after success.
- User cannot resend again until 60 seconds pass.

Step 12 - Add The Auth Gate Rule

For email/password accounts, the app must not treat the user as authenticated until email is verified.

At repository or controller boundary:

```
final user = authBackend.getCurrentUser();  
final canEnterApp = user != null && user.emailVerified;
```

Google accounts normally return verified emails through Google. If the app wants one rule for all providers, use `user.emailVerified` for all users.

Package Requirements

Use commands instead of hard-coding versions. This allows a future project to receive current compatible package versions.

```
flutter pub add firebase_core firebase_auth google_sign_in connectivity_plus  
shared_preferences  
dart pub global activate flutterfire_cli  
flutterfire configure
```

Optional profile database support:

```
flutter pub add cloud_firestore  
flutterfire configure
```

Do not add state-management packages for this backend module.

Firebase Console Checklist

In Firebase Console:

1. Create or select a Firebase project.
2. Add Android and iOS apps.
3. Run `flutterfire configure` from the Flutter project root.
4. Enable Authentication > Sign-in method > Email/Password.
5. Enable Authentication > Sign-in method > Google.
6. For Android Google sign-in, add the app SHA-1 and SHA-256 fingerprints.
7. Re-download Android/iOS config files or rerun `flutterfire configure` after provider changes.
8. Customize Authentication > Templates for verification email and password reset if needed.
9. If using custom continue URLs, add them to Authentication > Settings > Authorized domains.

Recommended Folder Structure

```
lib/  
|-- firebase_options.dart  
|-- main.dart  
|
```

```

|-- core/
| |-- errors/
| | |-- auth_exception.dart
| | +-- auth_failure.dart
| |-- network/
| | +-- network_info.dart
| |-- storage/
| | +-- remembered_email_storage.dart
| +-- utils/
| | |-- auth_action_code_settings.dart
| | |-- auth_cooldown.dart
| | |-- auth_validators.dart
| | +-- result.dart
|
+-- features/
  +-- authentication/
    |-- data/
    | |-- datasources/
    | | |-- auth_remote_data_source.dart
    | | +-- firebase_auth_remote_data_source.dart
    | |-- models/
    | | +-- user_model.dart
    | +-- repositories/
    | | +-- firebase_auth_repository.dart
    |
    |-- domain/
    | |-- entities/
    | | +-- user_entity.dart
    | |-- repositories/
    | | +-- auth_repository.dart
    | +-- usecases/
    | | |-- auth_usecases.dart
    | | |-- get_auth_state_changes_usecase.dart
    | | |-- get_current_user_usecase.dart
    | | |-- reload_and_check_email_verified_usecase.dart
    | | |-- remember_email_usecase.dart
    | | |-- send_password_reset_email_usecase.dart
    | | |-- send_verification_email_usecase.dart
    | | |-- sign_in_with_email_usecase.dart
    | | |-- sign_in_with_google_usecase.dart
    | | |-- sign_out_usecase.dart
    | | +-- sign_up_with_email_usecase.dart
    |
    +-- auth_backend_factory.dart

```

Firebase Initialization

The host app must initialize Firebase before using any auth backend class.

File: `lib/main.dart`

```

import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/widgets.dart';

import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  // runApp(...) belongs to the presentation app, not this backend guide.
}

```

Core Result Type

File: `lib/core/utils/result.dart`

```

sealed class Result<T, E> {
  const Result();

  bool get isSuccess => this is Success<T, E>;
  bool get isFailure => this is Failure<T, E>;
}

```

```

R when<R>({
  required R Function(T value) success,
  required R Function(E error) failure,
}) {
  final self = this;
  return switch (self) {
    Success<T, E>() => success(self.value),
    Failure<T, E>() => failure(self.error),
  };
}

final class Success<T, E> extends Result<T, E> {
  const Success(this.value);
  final T value;
}

final class Failure<T, E> extends Result<T, E> {
  const Failure(this.error);
  final E error;
}

```

Core Auth Exceptions

File: `lib/core/errors/auth_exception.dart`

```

import 'package:firebase_auth/firebase_auth.dart';

enum AuthExceptionCode {
  networkUnavailable,
  invalidEmail,
  emailAlreadyInUse,
  weakPassword,
  wrongPassword,
  userNotFound,
  userDisabled,
  operationNotAllowed,
  tooManyRequests,
  emailNotVerified,
  accountExistsWithDifferentCredential,
  credentialAlreadyInUse,
  invalidCredential,
  popupClosedByUser,
  requiresRecentLogin,
  missingUser,
  unknown,
}

final class AuthException implements Exception {
  const AuthException({
    required this.code,
    required this.message,
    this.cause,
    this.stackTrace,
  });

  final AuthExceptionCode code;
  final String message;
  final Object? cause;
  final StackTrace? stackTrace;

  @override
  String toString() => 'AuthException(${code.name}): $message';
}

AuthException mapFirebaseAuthException(
  FirebaseAuthException exception, [
  StackTrace? stackTrace,
]) {
  final mappedCode = switch (exception.code) {
    'invalid-email' => AuthExceptionCode.invalidEmail,
    'email-already-in-use' => AuthExceptionCode.emailAlreadyInUse,
    'weak-password' => AuthExceptionCode.weakPassword,
    'wrong-password' => AuthExceptionCode.wrongPassword,
    'invalid-credential' => AuthExceptionCode.invalidCredential,
    'user-not-found' => AuthExceptionCode.userNotFound,
    'user-disabled' => AuthExceptionCode.userDisabled,
    'operation-not-allowed' => AuthExceptionCode.operationNotAllowed,
    'too-many-requests' => AuthExceptionCode.tooManyRequests,
    'account-exists-with-different-credential' =>
      AuthExceptionCode.accountExistsWithDifferentCredential,
    'credential-already-in-use' => AuthExceptionCode.credentialAlreadyInUse,
    'requires-recent-login' => AuthExceptionCode.requiresRecentLogin,
  };
}

```

```

    _ => AuthExceptionCode.unknown,
  };

  return AuthException(
    code: mappedCode,
    message: _messageForCode(mappedCode),
    cause: exception,
    stackTrace: stackTrace,
  );
}

String _messageForCode(AuthExceptionCode code) {
  return switch (code) {
    AuthExceptionCode.networkUnavailable =>
      'No internet connection. Please check your network and try again.',
    AuthExceptionCode.invalidEmail => 'Please enter a valid email address.',
    AuthExceptionCode.emailAlreadyInUse =>
      'An account already exists for this email address.',
    AuthExceptionCode.weakPassword => 'Password is too weak.',
    AuthExceptionCode.wrongPassword => 'The password is incorrect.',
    AuthExceptionCode.userNotFound => 'No account was found for this email.',
    AuthExceptionCode.userDisabled => 'This account has been disabled.',
    AuthExceptionCode.operationNotAllowed =>
      'This sign-in method is not enabled.',
    AuthExceptionCode.tooManyRequests =>
      'Too many attempts. Please wait and try again.',
    AuthExceptionCode.emailNotVerified =>
      'Please verify your email before signing in.',
    AuthExceptionCode.accountExistsWithDifferentCredential =>
      'This email is already linked to another sign-in method.',
    AuthExceptionCode.credentialAlreadyInUse =>
      'This credential is already linked to another account.',
    AuthExceptionCode.invalidCredential =>
      'The sign-in credential is invalid or expired.',
    AuthExceptionCode.popupClosedByUser => 'Sign-in was cancelled.',
    AuthExceptionCode.requiresRecentLogin =>
      'Please sign in again before continuing.',
    AuthExceptionCode.missingUser => 'No authenticated user is available.',
    AuthExceptionCode.unknown =>
      'We could not complete the request. Please try again.',
  };
}

```

Domain Failures

File: `lib/core/errors/auth_failure.dart`

```

import 'auth_exception.dart';

enum AuthFailureCode {
  networkUnavailable,
  invalidEmail,
  emailAlreadyInUse,
  weakPassword,
  wrongPassword,
  userNotFound,
  userDisabled,
  operationNotAllowed,
  tooManyRequests,
  emailNotVerified,
  accountExistsWithDifferentCredential,
  credentialAlreadyInUse,
  invalidCredential,
  cancelled,
  requiresRecentLogin,
  missingUser,
  unknown,
}

final class AuthFailure {
  const AuthFailure({
    required this.code,
    required this.message,
  });

  final AuthFailureCode code;
  final String message;

  factory AuthFailure.fromException(AuthException exception) {
    return AuthFailure(
      code: switch (exception.code) {
        AuthExceptionCode.networkUnavailable =>
          AuthFailureCode.networkUnavailable,

```

```

    AuthExceptionCode.invalidEmail => AuthFailureCode.invalidEmail,
    AuthExceptionCode.emailAlreadyInUse =>
      AuthFailureCode.emailAlreadyInUse,
    AuthExceptionCode.weakPassword => AuthFailureCode.weakPassword,
    AuthExceptionCode.wrongPassword => AuthFailureCode.wrongPassword,
    AuthExceptionCode.userNotFound => AuthFailureCode.userNotFound,
    AuthExceptionCode.userDisabled => AuthFailureCode.userDisabled,
    AuthExceptionCode.operationNotAllowed =>
      AuthFailureCode.operationNotAllowed,
    AuthExceptionCode.tooManyRequests => AuthFailureCode.tooManyRequests,
    AuthExceptionCode.emailNotVerified =>
      AuthFailureCode.emailNotVerified,
    AuthExceptionCode.accountExistsWithDifferentCredential =>
      AuthFailureCode.accountExistsWithDifferentCredential,
    AuthExceptionCode.credentialAlreadyInUse =>
      AuthFailureCode.credentialAlreadyInUse,
    AuthExceptionCode.invalidCredential =>
      AuthFailureCode.invalidCredential,
    AuthExceptionCode.popupClosedByUser => AuthFailureCode.cancelled,
    AuthExceptionCode.requiresRecentLogin =>
      AuthFailureCode.requiresRecentLogin,
    AuthExceptionCode.missingUser => AuthFailureCode.missingUser,
    AuthExceptionCode.unknown => AuthFailureCode.unknown,
  },
  message: exception.message,
);
}

factory AuthFailure.unknown([Object? error]) {
  return const AuthFailure(
    code: AuthFailureCode.unknown,
    message: 'We could not complete the request. Please try again.',
  );
}
}

```

Validation Helpers

Keep validators independent from UI. Return error codes, not widget text.

File: `lib/core/utils/auth_validators.dart`

```

enum AuthValidationError {
  required,
  invalidEmail,
  passwordTooShort,
  nameTooShort,
  invalidPhone,
}

final class AuthValidators {
  const AuthValidators._();

  static final RegExp _emailRegex = RegExp(
    r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$',
  );

  static final RegExp _libyaPhoneRegex = RegExp(r'^(091|092|093|094)\d{7}$');

  static AuthValidationError? email(String value) {
    final trimmed = value.trim();
    if (trimmed.isEmpty) return AuthValidationError.required;
    if (!_emailRegex.hasMatch(trimmed)) return AuthValidationError.invalidEmail;
    return null;
  }

  static AuthValidationError? password(String value, {int minLength = 6}) {
    if (value.isEmpty) return AuthValidationError.required;
    if (value.length < minLength) return AuthValidationError.passwordTooShort;
    return null;
  }

  static AuthValidationError? fullName(String value) {
    final trimmed = value.trim();
    if (trimmed.isEmpty) return AuthValidationError.required;
    if (trimmed.length < 2) return AuthValidationError.nameTooShort;
    return null;
  }

  static AuthValidationError? libyaPhone(String value, {bool required = false}) {
    final trimmed = value.trim();

```

```

    if (trimmed.isEmpty) {
      return required ? AuthValidationError.required : null;
    }
    if (!_libyaPhoneRegex.hasMatch(trimmed)) {
      return AuthValidationError.invalidPhone;
    }
    return null;
  }

  static String normalizeEmail(String email) => email.trim().toLowerCase();
  static String normalizeName(String name) => name.trim();
  static String normalizePhone(String phone) => phone.trim();
}

```

Network Helper

Connectivity is not a perfect internet guarantee, but it prevents obvious offline calls.

File: `lib/core/network/network_info.dart`

```

import 'package:connectivity_plus/connectivity_plus.dart';

abstract interface class NetworkInfo {
  Future<bool> get hasConnection;
}

final class ConnectivityNetworkInfo implements NetworkInfo {
  ConnectivityNetworkInfo(this._connectivity);

  final Connectivity _connectivity;

  @override
  Future<bool> get hasConnection async {
    final results = await _connectivity.checkConnectivity();
    return !results.contains(ConnectivityResult.none);
  }
}

```

Remembered Email Storage

Remember-me should store only the email address, not the password.

File: `lib/core/storage/remembered_email_storage.dart`

```

import 'package:shared_preferences/shared_preferences.dart';

abstract interface class RememberedEmailStorage {
  Future<String?> readEmail();
  Future<void> saveEmail(String email);
  Future<void> clearEmail();
}

final class SharedPreferencesRememberedEmailStorage
  implements RememberedEmailStorage {
  SharedPreferencesRememberedEmailStorage(this._preferences);

  static const _key = 'auth.remembered_email';

  final SharedPreferences _preferences;

  @override
  Future<String?> readEmail() async {
    final email = _preferences.getString(_key);
    if (email == null || email.trim().isEmpty) return null;
    return email;
  }

  @override
  Future<void> saveEmail(String email) async {
    await _preferences.setString(_key, email.trim().toLowerCase());
  }

  @override
  Future<void> clearEmail() async {

```

```

    await _preferences.remove(_key);
  }
}

```

Action Code Settings Helper

Use this when the project needs Firebase email links to return to a website or deep link.

File: `lib/core/utils/auth_action_code_settings.dart`

```

import 'package:firebase_auth/firebase_auth.dart';

final class AuthActionCodeSettingsFactory {
  const AuthActionCodeSettingsFactory._();

  static ActionCodeSettings emailVerification({
    required String continueUrl,
    required String androidPackageName,
    required String iosBundleId,
    bool handleCodeInApp = true,
    bool androidInstallApp = true,
    String? androidMinimumVersion,
  }) {
    return ActionCodeSettings(
      url: continueUrl,
      handleCodeInApp: handleCodeInApp,
      androidPackageName: androidPackageName,
      androidInstallApp: androidInstallApp,
      androidMinimumVersion: androidMinimumVersion,
      iosBundleId: iosBundleId,
    );
  }

  static ActionCodeSettings passwordReset({
    required String continueUrl,
    required String androidPackageName,
    required String iosBundleId,
    bool handleCodeInApp = true,
    bool androidInstallApp = true,
    String? androidMinimumVersion,
  }) {
    return ActionCodeSettings(
      url: continueUrl,
      handleCodeInApp: handleCodeInApp,
      androidPackageName: androidPackageName,
      androidInstallApp: androidInstallApp,
      androidMinimumVersion: androidMinimumVersion,
      iosBundleId: iosBundleId,
    );
  }
}

```

Cooldown Helper For Resend Buttons

This is backend-safe pure Dart. The UI or controller can use it to disable resend actions for 60 seconds.

File: `lib/core/utils/auth_cooldown.dart`

```

final class AuthCooldown {
  AuthCooldown({this.duration = const Duration(seconds: 60)});

  final Duration duration;
  DateTime? _lastTriggeredAt;

  bool get canRun => remaining == Duration.zero;

  Duration get remaining {
    final last = _lastTriggeredAt;
    if (last == null) return Duration.zero;

    final elapsed = DateTime.now().difference(last);
    if (elapsed >= duration) return Duration.zero;
    return duration - elapsed;
  }
}

```

```

void markTriggered([DateTime? now]) {
  _lastTriggeredAt = now ?? DateTime.now();
}

void reset() {
  _lastTriggeredAt = null;
}
}

```

Domain Entity

File: `lib/features/authentication/domain/entities/user_entity.dart`

```

final class UserEntity {
  const UserEntity({
    required this.uid,
    required this.email,
    required this.emailVerified,
    this.displayName,
    this.photoUrl,
    this.phoneNumber,
    this.providerIds = const <String>[],
  });

  final String uid;
  final String email;
  final bool emailVerified;
  final String? displayName;
  final String? photoUrl;
  final String? phoneNumber;
  final List<String> providerIds;
}

```

Repository Contract

File: `lib/features/authentication/domain/repositories/auth_repository.dart`

```

import 'package:firebase_auth/firebase_auth.dart';

import '../../../core/errors/auth_failure.dart';
import '../../../core/utils/result.dart';
import '../entities/user_entity.dart';

abstract interface class IAuthRepository {
  Stream<UserEntity?> authStateChanges();

  UserEntity? get currentUser;

  Future<Result<UserEntity, AuthFailure>> signUpWithEmail({
    required String email,
    required String password,
    required String fullName,
   ActionCodeSettings? emailVerificationSettings,
  });

  Future<Result<UserEntity, AuthFailure>> signInWithEmail({
    required String email,
    required String password,
  });

  Future<Result<UserEntity, AuthFailure>> signInWithGoogle();

  Future<Result<void, AuthFailure>> sendVerificationEmail({
    ActionCodeSettings? settings,
  });

  Future<Result<bool, AuthFailure>> reloadAndCheckEmailVerified();

  Future<Result<void, AuthFailure>> sendPasswordResetEmail({
    required String email,
    ActionCodeSettings? settings,
  });

  Future<Result<String?, AuthFailure>> getIdToken({bool forceRefresh = false});

  Future<Result<void, AuthFailure>> signOut();
}

```


}

Firestore User Model

File: lib/features/authentication/data/models/user_model.dart

```
import 'package:firebase_auth/firebase_auth.dart';

import '../domain/entities/user_entity.dart';

final class UserModel extends UserEntity {
  const UserModel({
    required super.uid,
    required super.email,
    required super.emailVerified,
    super.displayName,
    super.photoUrl,
    super.phoneNumber,
    super.providerIds,
  });

  factory UserModel.fromFirebaseUser(User user) {
    return UserModel(
      uid: user.uid,
      email: user.email ?? '',
      emailVerified: user.emailVerified,
      displayName: user.displayName,
      photoUrl: user.photoURL,
      phoneNumber: user.phoneNumber,
      providerIds: user.providerData.map((info) => info.providerId).toList(),
    );
  }
}
```

Remote Data Source Contract

File: lib/features/authentication/data/datasources/auth_remote_data_source.dart

```
import 'package:firebase_auth/firebase_auth.dart';

import '../models/user_model.dart';

abstract interface class AuthRemoteDataSource {
  Stream<UserModel?> authStateChanges();

  UserModel? get currentUser;

  Future<UserModel> signUpWithEmail({
    required String email,
    required String password,
    required String fullName,
    ActionCodeSettings? emailVerificationSettings,
  });

  Future<UserModel> signInWithEmail({
    required String email,
    required String password,
  });

  Future<UserModel> signInWithGoogle();

  Future<void> sendVerificationEmail({ActionCodeSettings? settings});

  Future<bool> reloadAndCheckEmailVerified();

  Future<void> sendPasswordResetEmail({
    required String email,
    ActionCodeSettings? settings,
  });

  Future<String?> getIdToken({bool forceRefresh = false});

  Future<void> signOut();
}
```

Firestore Remote Data Source

File: lib/features/authentication/data/datasources/firestore_auth_remote_data_source.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';
import 'package:google_sign_in/google_sign_in.dart';

import '../../../core/errors/auth_exception.dart';
import '../../../core/utils/auth_validators.dart';
import '../../../models/user_model.dart';
import 'auth_remote_data_source.dart';

final class FirebaseAuthRemoteDataSource implements AuthRemoteDataSource {
  FirebaseAuthRemoteDataSource({
    required FirebaseAuth firebaseAuth,
  }) : _firebaseAuth = firebaseAuth;

  final FirebaseAuth _firebaseAuth;

  @override
  Stream<UserModel?> authStateChanges() {
    return _firebaseAuth.authStateChanges().map((user) {
      if (user == null) return null;
      return UserModel.fromFirebaseUser(user);
    });
  }

  @override
  UserModel? get currentUser {
    final user = _firebaseAuth.currentUser;
    if (user == null) return null;
    return UserModel.fromFirebaseUser(user);
  }

  @override
  Future<UserModel> signUpWithEmail({
    required String email,
    required String password,
    required String fullName,
   ActionCodeSettings? emailVerificationSettings,
  }) async {
    return _guard(() async {
      final credential = await _firebaseAuth.createUserWithEmailAndPassword(
        email: AuthValidators.normalizeEmail(email),
        password: password,
      );

      final user = credential.user;
      if (user == null) {
        throw const AuthException(
          code: AuthExceptionCode.missingUser,
          message: 'No authenticated user is available.',
        );
      }

      await user.updateDisplayName(AuthValidators.normalizeName(fullName));

      if (emailVerificationSettings == null) {
        await user.sendEmailVerification();
      } else {
        await user.sendEmailVerification(emailVerificationSettings);
      }

      await user.reload();
      final refreshedUser = _firebaseAuth.currentUser ?? user;
      return UserModel.fromFirebaseUser(refreshedUser);
    });
  }

  @override
  Future<UserModel> signInWithEmail({
    required String email,
    required String password,
  }) async {
    return _guard(() async {
      final credential = await _firebaseAuth.signInWithEmailAndPassword(
        email: AuthValidators.normalizeEmail(email),
        password: password,
      );

      final user = credential.user;
      if (user == null) {
        throw const AuthException(
          code: AuthExceptionCode.missingUser,

```

```

        message: 'No authenticated user is available.',
    );
}

await user.reload();
final refreshedUser = _firebaseAuth.currentUser ?? user;

if (!refreshedUser.emailVerified) {
    await _firebaseAuth.signOut();
    throw const AuthException(
        code: AuthExceptionCode.emailNotVerified,
        message: 'Please verify your email before signing in.',
    );
}

return UserModel.fromFirebaseUser(refreshedUser);
});
}

@override
Future<UserModel> signInWithGoogle() async {
    return _guard(() async {
        UserCredential credential;

        if (kIsWeb) {
            final provider = GoogleAuthProvider();
            credential = await _firebaseAuth.signInWithPopup(provider);
        } else {
            final googleUser = await GoogleSignIn.instance.authenticate();
            final googleAuth = googleUser.authentication;

            final firebaseCredential = GoogleAuthProvider.credential(
                idToken: googleAuth.idToken,
            );

            credential = await _firebaseAuth.signInWithCredential(
                firebaseCredential,
            );
        }

        final user = credential.user;
        if (user == null) {
            throw const AuthException(
                code: AuthExceptionCode.missingUser,
                message: 'No authenticated user is available.',
            );
        }

        return UserModel.fromFirebaseUser(user);
    });
}

@override
Future<void> sendVerificationEmail({ActionCodeSettings? settings}) async {
    return _guard(() async {
        final user = _firebaseAuth.currentUser;
        if (user == null) {
            throw const AuthException(
                code: AuthExceptionCode.missingUser,
                message: 'No authenticated user is available.',
            );
        }

        if (settings == null) {
            await user.sendEmailVerification();
        } else {
            await user.sendEmailVerification(settings);
        }
    });
}

@override
Future<bool> reloadAndCheckEmailVerified() async {
    return _guard(() async {
        final user = _firebaseAuth.currentUser;
        if (user == null) {
            throw const AuthException(
                code: AuthExceptionCode.missingUser,
                message: 'No authenticated user is available.',
            );
        }

        await user.reload();
        return _firebaseAuth.currentUser?.emailVerified ?? false;
    });
}

@override
Future<void> sendPasswordResetEmail({

```

```

    required String email,
    ActionCodeSettings? settings,
  }) async {
    return _guard(() async {
      if (settings == null) {
        await _firebaseAuth.sendPasswordResetEmail(
          email: AuthValidators.normalizeEmail(email),
        );
      } else {
        await _firebaseAuth.sendPasswordResetEmail(
          email: AuthValidators.normalizeEmail(email),
          actionCodeSettings: settings,
        );
      }
    });
  }

  @override
  Future<String?> getIdToken({bool forceRefresh = false}) async {
    return _guard(() async {
      final user = _firebaseAuth.currentUser;
      if (user == null) return null;
      return user.getIdToken(forceRefresh);
    });
  }

  @override
  Future<void> signOut() async {
    return _guard(() async {
      if (!kIsWeb) {
        try {
          await GoogleSignIn.instance.signOut();
        } catch (_) {
          // Firebase sign-out below is the source of truth.
        }
      }
      await _firebaseAuth.signOut();
    });
  }

  Future<T> _guard<T>(Future<T> Function() body) async {
    try {
      return await body();
    } on FirebaseAuthException catch (error, stackTrace) {
      throw mapFirebaseAuthException(error, stackTrace);
    } on AuthException {
      rethrow;
    } catch (error, stackTrace) {
      throw AuthException(
        code: AuthExceptionCode.unknown,
        message: 'We could not complete the request. Please try again.',
        cause: error,
        stackTrace: stackTrace,
      );
    }
  }
}

```

Repository Implementation

File: `lib/features/authentication/data/repositories/firebase_auth_repository.dart`

```

import 'package:firebase_auth/firebase_auth.dart';

import '../../../core/errors/auth_exception.dart';
import '../../../core/errors/auth_failure.dart';
import '../../../core/network/network_info.dart';
import '../../../core/utils/result.dart';
import '../../../domain/entities/user_entity.dart';
import '../../../domain/repositories/auth_repository.dart';
import '../../../datasources/auth_remote_data_source.dart';

final class FirebaseAuthRepository implements IAuthRepository {
  FirebaseAuthRepository({
    required AuthRemoteDataSource remoteDataSource,
    required NetworkInfo networkInfo,
  }) : _remoteDataSource = remoteDataSource,
       _networkInfo = networkInfo;

  final AuthRemoteDataSource _remoteDataSource;
  final NetworkInfo _networkInfo;

  @override

```

```

Stream<UserEntity?> authStateChanges() => _remoteDataSource.authStateChanges();

@override
UserEntity? get currentUser => _remoteDataSource.currentUser;

@override
Future<Result<UserEntity, AuthFailure>> signUpWithEmail({
  required String email,
  required String password,
  required String fullName,
 ActionCodeSettings? emailVerificationSettings,
}) {
  return _guard(
    () => _remoteDataSource.signUpWithEmail(
      email: email,
      password: password,
      fullName: fullName,
      emailVerificationSettings: emailVerificationSettings,
    ),
  );
}

@override
Future<Result<UserEntity, AuthFailure>> signInWithEmail({
  required String email,
  required String password,
}) {
  return _guard(
    () => _remoteDataSource.signInWithEmail(
      email: email,
      password: password,
    ),
  );
}

@override
Future<Result<UserEntity, AuthFailure>> signInWithGoogle() {
  return _guard(_remoteDataSource.signInWithGoogle);
}

@override
Future<Result<void, AuthFailure>> sendVerificationEmail({
  ActionCodeSettings? settings,
}) {
  return _guard(
    () => _remoteDataSource.sendVerificationEmail(settings: settings),
  );
}

@override
Future<Result<bool, AuthFailure>> reloadAndCheckEmailVerified() {
  return _guard(_remoteDataSource.reloadAndCheckEmailVerified);
}

@override
Future<Result<void, AuthFailure>> sendPasswordResetEmail({
  required String email,
  ActionCodeSettings? settings,
}) {
  return _guard(
    () => _remoteDataSource.sendPasswordResetEmail(
      email: email,
      settings: settings,
    ),
  );
}

@override
Future<Result<String?, AuthFailure>> getIdToken({
  bool forceRefresh = false,
}) {
  return _guard(
    () => _remoteDataSource.getIdToken(forceRefresh: forceRefresh),
    requiresNetwork: false,
  );
}

@override
Future<Result<void, AuthFailure>> signOut() {
  return _guard(_remoteDataSource.signOut, requiresNetwork: false);
}

Future<Result<T, AuthFailure>> _guard<T>(
  Future<T> Function() body, {
  bool requiresNetwork = true,
}) async {
  if (requiresNetwork && !await _networkInfo.hasConnection) {
    return Failure(
      AuthFailure.fromException(

```

```

        const AuthException(
          code: AuthExceptionCode.networkUnavailable,
          message:
            'No internet connection. Please check your network and try again.',
        ),
      ),
    );
  }

  try {
    return Success(await body());
  } on AuthException catch (error) {
    return Failure(AuthFailure.fromException(error));
  } catch (error) {
    return Failure(AuthFailure.unknown(error));
  }
}
}

```

Use Cases

Each use case should be small and framework-agnostic. The presentation layer can call these from any state-management choice later.

File: `lib/features/authentication/domain/usecases/sign_in_with_email_usecase.dart`

```

import '../../../core/errors/auth_failure.dart';
import '../../../core/utils/result.dart';
import '../entities/user_entity.dart';
import '../repositories/auth_repository.dart';

final class SignInWithEmailParams {
  const SignInWithEmailParams({
    required this.email,
    required this.password,
  });

  final String email;
  final String password;
}

final class SignInWithEmailUseCase {
  const SignInWithEmailUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<UserEntity, AuthFailure>> call(
    SignInWithEmailParams params,
  ) {
    return _repository.signInWithEmail(
      email: params.email,
      password: params.password,
    );
  }
}

```

File: `lib/features/authentication/domain/usecases/sign_up_with_email_usecase.dart`

```

import 'package:firebase_auth/firebase_auth.dart';

import '../../../core/errors/auth_failure.dart';
import '../../../core/utils/result.dart';
import '../entities/user_entity.dart';
import '../repositories/auth_repository.dart';

final class SignUpWithEmailParams {
  const SignUpWithEmailParams({
    required this.email,
    required this.password,
    required this.fullName,
    this.emailVerificationSettings,
  });

  final String email;
  final String password;
  final String fullName;
  final ActionCodeSettings? emailVerificationSettings;
}

```

```

final class SignUpWithEmailUseCase {
  const SignUpWithEmailUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<UserEntity, AuthFailure>> call(
    SignUpWithEmailParams params,
  ) {
    return _repository.signUpWithEmail(
      email: params.email,
      password: params.password,
      fullName: params.fullName,
      emailVerificationSettings: params.emailVerificationSettings,
    );
  }
}

```

File: lib/features/authentication/domain/usecases/sign_in_with_google_usecase.dart

```

import '../../../../../core/errors/auth_failure.dart';
import '../../../../../core/utils/result.dart';
import '../entities/user_entity.dart';
import '../repositories/auth_repository.dart';

final class SignInWithGoogleUseCase {
  const SignInWithGoogleUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<UserEntity, AuthFailure>> call() {
    return _repository.signInWithGoogle();
  }
}

```

File: lib/features/authentication/domain/usecases/send_verification_email_usecase.dart

```

import 'package:firebase_auth/firebase_auth.dart';

import '../../../../../core/errors/auth_failure.dart';
import '../../../../../core/utils/result.dart';
import '../repositories/auth_repository.dart';

final class SendVerificationEmailUseCase {
  const SendVerificationEmailUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<void, AuthFailure>> call({
    ActionCodeSettings? settings,
  }) {
    return _repository.sendVerificationEmail(settings: settings);
  }
}

```

File: lib/features/authentication/domain/usecases/reload_and_check_email_verified_usecase.dart

```

import '../../../../../core/errors/auth_failure.dart';
import '../../../../../core/utils/result.dart';
import '../repositories/auth_repository.dart';

final class ReloadAndCheckEmailVerifiedUseCase {
  const ReloadAndCheckEmailVerifiedUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<bool, AuthFailure>> call() {
    return _repository.reloadAndCheckEmailVerified();
  }
}

```

File: lib/features/authentication/domain/usecases/send_password_reset_email_usecase.dart

```

import 'package:firebase_auth/firebase_auth.dart';

import '../../../../../core/errors/auth_failure.dart';
import '../../../../../core/utils/result.dart';
import '../repositories/auth_repository.dart';

final class SendPasswordResetEmailUseCase {

```

```

const SendPasswordResetEmailUseCase(this._repository);

final IAuthRepository _repository;

Future<Result<void, AuthFailure>> call({
  required String email,
  ActionCodeSettings? settings,
}) {
  return _repository.sendPasswordResetEmail(
    email: email,
    settings: settings,
  );
}

```

File: `lib/features/authentication/domain/usecases/sign_out_usecase.dart`

```

import '../../../core/errors/auth_failure.dart';
import '../../../core/utils/result.dart';
import '../repositories/auth_repository.dart';

final class SignOutUseCase {
  const SignOutUseCase(this._repository);

  final IAuthRepository _repository;

  Future<Result<void, AuthFailure>> call() {
    return _repository.signOut();
  }
}

```

File: `lib/features/authentication/domain/usecases/get_current_user_usecase.dart`

```

import '../entities/user_entity.dart';
import '../repositories/auth_repository.dart';

final class GetCurrentUserUseCase {
  const GetCurrentUserUseCase(this._repository);

  final IAuthRepository _repository;

  UserEntity? call() => _repository.currentUser;
}

```

File: `lib/features/authentication/domain/usecases/get_auth_state_changes_usecase.dart`

```

import '../entities/user_entity.dart';
import '../repositories/auth_repository.dart';

final class GetAuthStateChangesUseCase {
  const GetAuthStateChangesUseCase(this._repository);

  final IAuthRepository _repository;

  Stream<UserEntity?> call() => _repository.authStateChanges();
}

```

File: `lib/features/authentication/domain/usecases/remember_email_usecase.dart`

```

import '../../../core/storage/remembered_email_storage.dart';
import '../../../core/utils/auth_validators.dart';

final class RememberEmailUseCase {
  const RememberEmailUseCase(this._storage);

  final RememberedEmailStorage _storage;

  Future<void> saveIfRequested({
    required String email,
    required bool remember,
  }) async {
    if (remember) {
      await _storage.saveEmail(AuthValidators.normalizeEmail(email));
    } else {
      await _storage.clearEmail();
    }
  }

  Future<String?> readRememberedEmail() => _storage.readEmail();
}

```


File: lib/features/authentication/domain/usecases/auth_usecases.dart

```
export 'get_auth_state_changes_usecase.dart';
export 'get_current_user_usecase.dart';
export 'reload_and_check_email_verified_usecase.dart';
export 'remember_email_usecase.dart';
export 'send_password_reset_email_usecase.dart';
export 'send_verification_email_usecase.dart';
export 'sign_in_with_email_usecase.dart';
export 'sign_in_with_google_usecase.dart';
export 'sign_out_usecase.dart';
export 'sign_up_with_email_usecase.dart';
```

Backend Factory

Use this factory from the app composition root. It is not state management.

File: lib/features/authentication/auth_backend_factory.dart

```
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../../../core/network/network_info.dart';
import '../../../core/storage/remembered_email_storage.dart';
import 'data/datasources/firebase_auth_remote_data_source.dart';
import 'data/repositories/firebase_auth_repository.dart';
import 'domain/repositories/auth_repository.dart';
import 'domain/usecases/auth_usecases.dart';

final class AuthBackend {
  const AuthBackend({
    required this.repository,
    required this.signInWithEmail,
    required this.signUpWithEmail,
    required this.signInWithGoogle,
    required this.sendVerificationEmail,
    required this.reloadAndCheckEmailVerified,
    required this.sendPasswordResetEmail,
    required this.signOut,
    required this.getCurrentUser,
    required this.getAuthStateChanges,
    required this.rememberEmail,
  });

  final IAuthRepository repository;
  final SignInWithEmailUseCase signInWithEmail;
  final SignUpWithEmailUseCase signUpWithEmail;
  final SignInWithGoogleUseCase signInWithGoogle;
  final SendVerificationEmailUseCase sendVerificationEmail;
  final ReloadAndCheckEmailVerifiedUseCase reloadAndCheckEmailVerified;
  final SendPasswordResetEmailUseCase sendPasswordResetEmail;
  final SignOutUseCase signOut;
  final GetCurrentUserUseCase getCurrentUser;
  final GetAuthStateChangesUseCase getAuthStateChanges;
  final RememberEmailUseCase rememberEmail;
}

final class AuthBackendFactory {
  const AuthBackendFactory._();

  static Future<AuthBackend> create() async {
    final preferences = await SharedPreferences.getInstance();

    final remoteDataSource = FirebaseAuthRemoteDataSource(
      firebaseAuth: FirebaseAuth.instance,
    );

    final networkInfo = ConnectivityNetworkInfo(Connectivity());

    final repository = FirebaseAuthRepository(
      remoteDataSource: remoteDataSource,
      networkInfo: networkInfo,
    );

    final rememberedEmailStorage =
      SharedPreferencesRememberedEmailStorage(preferences);

    return AuthBackend(
      repository: repository,
```

```

    signInWithEmail: SignInWithEmailUseCase(repository),
    signUpWithEmail: SignUpWithEmailUseCase(repository),
    signInWithGoogle: SignInWithGoogleUseCase(repository),
    sendVerificationEmail: SendVerificationEmailUseCase(repository),
    reloadAndCheckEmailVerified:
      ReloadAndCheckEmailVerifiedUseCase(repository),
    sendPasswordResetEmail: SendPasswordResetEmailUseCase(repository),
    signOut: SignOutUseCase(repository),
    getCurrentUser: GetCurrentUserUseCase(repository),
    getAuthStateChanges: GetAuthStateChangesUseCase(repository),
    rememberEmail: RememberEmailUseCase(rememberedEmailStorage),
  );
}
}

```

Example Backend Usage From A Future Controller

This is not UI and not state management. It shows how a future controller or provider should call the backend.

```

final authBackend = await AuthBackendFactory.create();

final result = await authBackend.signInWithEmail(
  const SignInWithEmailParams(
    email: 'customer@example.com',
    password: 'secret123',
  ),
);

result.when(
  success: (user) {
    // Presentation decides where to navigate.
  },
  failure: (failure) {
    // Presentation decides how to show failure.message.
  },
);

```

Email Verification Flow

Backend rules:

- Sign-up creates the Firebase user.
- The backend immediately sends a verification email.
- The user is signed in by Firebase after account creation, but the app should treat the account as pending verification.
- Email/password sign-in must be blocked until `emailVerified` is true.
- Use `ReloadAndCheckEmailVerifiedUseCase` when the app returns from the email link or when polling.
- Use `AuthCooldown` to prevent resend spam.

Example polling logic outside the backend:

```

Future<bool> waitUntilVerified(AuthBackend authBackend) async {
  for (var i = 0; i < 20; i++) {
    await Future<void>.delayed(const Duration(seconds: 3));

    final result = await authBackend.reloadAndCheckEmailVerified();
    final verified = result.when(
      success: (value) => value,
      failure: (_) => false,
    );

    if (verified) return true;
  }

  return false;
}

```

Password Reset Flow

Backend rules:

- Validate email before calling the use case.
- Call `SendPasswordResetEmailUseCase`.
- Firebase sends the reset email.
- Firebase Console can customize the email template.
- A custom reset page can use Firebase action codes, but that page is outside this backend module.

```
final result = await authBackend.sendPasswordResetEmail(  
  email: 'customer@example.com',  
);
```

Optional Firestore Profile Sync

Only add this if the project needs a user document in Firestore. Keep it separate from Firebase Auth.

Recommended collection:

```
users/{uid}
```

Recommended fields:

```
uid: string  
email: string  
displayName: string?  
photoUrl: string?  
phoneNumber: string?  
emailVerified: boolean  
providerIds: array<string>  
createdAt: timestamp  
updatedAt: timestamp
```

Optional profile sync service:

```
import 'package:cloud_firestore/cloud_firestore.dart';  
  
import '../domain/entities/user_entity.dart';  
  
final class FirebaseUserProfileSyncService {  
  FirebaseUserProfileSyncService(this._firestore);  
  
  final FirebaseFirestore _firestore;  
  
  Future<void> upsertUserProfile(UserEntity user) async {  
    final ref = _firestore.collection('users').doc(user.uid);  
  
    await ref.set(  
      {  
        'uid': user.uid,  
        'email': user.email,  
        'displayName': user.displayName,  
        'photoUrl': user.photoUrl,  
        'phoneNumber': user.phoneNumber,  
        'emailVerified': user.emailVerified,  
        'providerIds': user.providerIds,  
        'updatedAt': FieldValue.serverTimestamp(),  
        'createdAt': FieldValue.serverTimestamp(),  
      },  
      SetOptions(merge: true),  
    );  
  }  
}
```

Firestore security rules should allow only the authenticated owner to read/write their own document:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, update, delete: if request.auth != null
        && request.auth.uid == userId;
      allow create: if request.auth != null
        && request.auth.uid == userId
        && request.resource.data.uid == request.auth.uid;
    }
  }
}
```

Implementation Checklist For The AI

Use this checklist when implementing in a new project.

- ☐ Run `flutter pub add firebase_core firebase_auth google_sign_in connectivity_plus shared_preferences`.
- ☐ Run `dart pub global activate flutterfire_cli`.
- ☐ Run `flutterfire configure`.
- ☐ Initialize Firebase before using the auth backend.
- ☐ Create the folder structure from this guide.
- ☐ Add `Result`.
- ☐ Add `AuthException` and Firebase exception mapper.
- ☐ Add `AuthFailure`.
- ☐ Add `AuthValidators`.
- ☐ Add `NetworkInfo`.
- ☐ Add `RememberedEmailStorage`.
- ☐ Add `AuthActionCodeSettingsFactory`.
- ☐ Add `AuthCooldown`.
- ☐ Add `UserEntity`.
- ☐ Add `UserModel`.
- ☐ Add `IAuthRepository`.
- ☐ Add `AuthRemoteDataSource`.
- ☐ Add `FirebaseAuthRemoteDataSource`.
- ☐ Add `FirebaseAuthRepository`.
- ☐ Add all use cases.
- ☐ Add `AuthBackendFactory`.
- ☐ Enable Email/Password and Google sign-in in Firebase Console.
- ☐ Add Android SHA fingerprints for Google sign-in.
- ☐ Verify that email/password sign-in rejects unverified accounts.
- ☐ Verify that Google sign-in returns a `UserEntity`.
- ☐ Verify that sign-out signs out Firebase and Google.

- [] Do not create UI, widgets, routes, providers, controllers, or tests unless explicitly requested.

Common Mistakes To Avoid

- Do not place Firebase SDK calls in presentation code.
- Do not store passwords locally.
- Do not treat `currentUser` as initialized on app startup; use `authStateChanges`.
- Do not allow email/password users into the app before email verification if the project requires verified email.
- Do not use `authStateChanges().listen(...)` inside widget build methods.
- Do not map raw Firebase exception strings directly to UI.
- Do not force a state-management solution into this backend module.
- Do not mix Firestore profile sync into the Firebase Auth data source unless the project explicitly asks for it.
- Do not forget Android SHA-1/SHA-256 for Google sign-in.
- Do not use generated dependency injection unless the project already uses it.

Final Handoff Contract

After implementing this backend module, the AI should report:

1. The files created.
2. The Firebase providers the developer must enable.
3. Whether Firebase initialization was already present or added.
4. How the presentation layer should call the use cases.
5. Any platform setup still required, such as Android SHA fingerprints or iOS bundle configuration.